

# SACI: Explainable Graph-Based Attack Surface Visibility Scoring Using SIEM Telemetry and Threat Intelligence

Ismet Arslan

**Abstract**—This study presents SACI, a scope-bounded framework for measuring security information and event management visibility through evidence relationships among assets, log sources, detection controls, Wazuh rules, MITRE ATT&CK techniques, and cyber-threat-intelligence objects. The method combines Criticality-Weighted Log Coverage (CWLC), Control/Alert Coverage (CAC), in-scope ATT&CK Detection Coverage (MDC), CTI flag closure (CTIC), and Telemetry Freshness (TF) using active-metric weight normalization. In the final publication snapshot of a controlled six-asset Wazuh-MISP laboratory, 12/12 expected asset-log-source pairs, 25/25 enabled controls, 13/13 in-scope techniques, and 8/8 CTI flags across two IOC records were closed; all five components and the composite SACI score were 100. The published graph contains 97 declared nodes and 171 edge rows with observed=1; an integrity audit additionally identified 165 unique relationship triples, two undeclared endpoints, and six isolated nodes. A separately maintained 20-scenario historical suite exhibited directionally expected score changes under component loss and recovery. The laboratory topology and evidence paths are auditable, but exact software versions and complete environment provisioning were not preserved. The results should be interpreted as evidence closure within a predefined laboratory scope, not as a guarantee of operational security.

**Keywords**—SACI, SIEM, Wazuh, evidence graph, MITRE ATT&CK, cyber threat intelligence, visibility scoring, auditability

## I. INTRODUCTION

Modern security operations centers (SOCs) continuously collect endpoint, identity, network, and application telemetry to detect malicious behavior and prioritize response. The presence of a SIEM, however, does not by itself demonstrate that critical assets are monitored by expected sources or that relevant adversary techniques have observable evidence.

The growth of enterprise attack surfaces makes this distinction increasingly important. Cloud services, remote-access platforms, firewalls, identity providers, and third-party integrations produce fragmented evidence. High alert volume may coexist with stale telemetry, disabled controls, or disconnected technique mappings that create operational blind spots.

This study asks: how can heterogeneous SIEM telemetry and threat context be transformed into a visibility score with traceable provenance and an explicitly bounded scope? SACI joins assets, log sources, detection controls, rules, ATT&CK techniques, and CTI objects in a typed evidence graph.

SACI is not a model of risk, attack prevention, or probability of detection. It deterministically reports which expected relationships close within a declared evidence

universe, how each component contributes to the score, and which data-integrity exceptions constrain interpretation.

**The contributions are fivefold: (i) a typed graph schema for SIEM-CTI evidence; (ii) an implemented score combining CWLC, CAC, MDC, CTIC, and TF with active-weight renormalization for N/A metrics; (iii) a final publication snapshot plus a separately maintained 20-scenario historical sensitivity suite; (iv) a laboratory setup that separates evidence-verified roles from the requirements for rebuilding Wazuh, MISP, pfSense, Windows, Linux, Sysmon, PowerShell, and Ansible components; and (v) an auditable research companion publishing raw outputs, the scorer, graph builder, and integrity checks [13].**

## II. RELATED WORK

### A. SIEM Telemetry and Continuous Monitoring

Continuous monitoring requires security status to be evaluated through evidence observed over time rather than isolated checks [4]. NIST CSF 2.0 likewise connects governance and measurement to organizational context [5]. SACI applies this principle to asset-log-source, control-rule, and technique relationships.

Wazuh provides endpoint monitoring, rule-based detection, and ATT&CK-related rule metadata [2]. A technique identifier in a rule, however, does not prove that the corresponding asset and log source were observed or that the detection is externally effective. SACI therefore distinguishes configuration presence from observed coverage relationships.

### B. MITRE ATT&CK and Threat Intelligence

MITRE ATT&CK provides a structured cross-platform knowledge base of adversary tactics and techniques [1]. Despite broad adoption, operationalizing ATT&CK coverage and interpreting assessment results remain open challenges [6]. SACI's MDC component counts only a predefined in-scope technique set.

STIX and TAXII support machine-readable representation and exchange of indicators and related cyber-threat objects [3], [11]. CTI models facilitate contextual enrichment, but the link between external context and internal telemetry still requires validation [10]. SACI exposes this link through four implemented flags.

### C. Graph-Based Cybersecurity Assessment

Cybersecurity knowledge graphs combine multi-source asset, event, vulnerability, and intelligence data for provenance and decision support [8]. Closed-world

assumptions and limited laboratory data can nevertheless make it difficult to distinguish a truly absent relation from missing evidence [9].

ATT&CK-based rating frameworks summarize assessment results through technique coverage [7]. Advanced persistent-threat studies also show that multistage behavior cannot be reduced to one indicator [12]. SACI complements this literature by measuring scope-bounded SIEM evidence closure rather than detection efficacy.

### III. PROPOSED MODEL

SACI contains five logical layers: scope and asset inventory, telemetry/detection evidence, ATT&CK-CTI context, typed graph fusion, and active-weight-normalized scoring. Fig. 1 summarizes how final-publication expectations and observations become scores, integrity findings, and publishable artifacts.

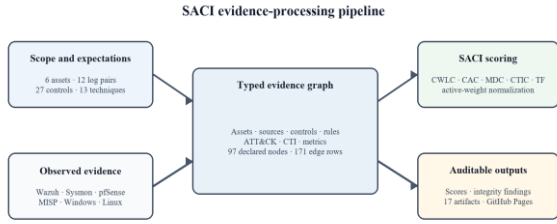


Fig. 1. SACI evidence-processing and publication pipeline.

#### A. Graph Schema

The graph is  $G=(V,E)$ .  $V$  contains asset, log-source, detection-control, Wazuh-rule, ATT&CK-technique, typed-CTI-object, platform, integration, score-metric, and reason-code nodes.  $E$  contains directed, typed evidence rows identified by source and target.

The final publication graph instantiates 17 relationship labels: alerted\_in, collected\_by, contains, contains\_ioc, contributes\_to, converted\_to\_alert, detects\_technique, emits, mapped\_to, maps\_to, matches\_ioc, produces\_rule, protected\_by, queries, triggers\_integration, uses, and uses\_log\_source. observed=1 denotes a row closed in the generated evidence table; observed=0 denotes a declared but unclosed row.

**Asset node:** an in-scope system identifier, hostname, and criticality value.

**Log-source/control node:** an expected telemetry source and enabled detection control supported by that evidence.

**Rule/technique node:** a Wazuh rule and its in-scope ATT&CK technique mapping.

**CTI/platform/metric node:** an IOC, MISP-Wazuh integration element, or computed SACI component.

**Integrity evidence:** observation state, unique triples, undeclared endpoints, isolated nodes, and conditional reason-code outputs are evaluated together.

#### B. Visibility Metrics

Component scores lie in  $[0,100]$ . For the applicable metric set  $M^*$ , the composite score is divided by the sum of active weights; N/A metrics are not assigned zero but are removed from both numerator and denominator.

$$SACI = \frac{\sum_{m \in M^*} w_m s_m}{\sum_{m \in M^*} w_m}$$

All metrics are active in the final evaluation, with  $CWLC=0.30$ ,  $CAC=0.25$ ,  $MDC=0.20$ ,  $CTIC=0.15$ , and  $TF=0.10$ . The weights sum to 1.00; they are design choices rather than probabilities.

$$L_a = \frac{\sum_{s \in S_a} w_{as} o_{as}}{\sum_{s \in S_a} w_{as}}$$

$$CWLC = 100 \frac{\sum_{a \in A^*} c_a L_a}{\sum_{a \in A^*} c_a}$$

Observation state and source weight are defined at log-source level; asset visibility is weighted by criticality.

$$CAC = 100 \frac{\sum_{c \in C} w_c e_c s_c}{\sum_{c \in C} w_c e_c}$$

Only enabled controls enter the numerator and denominator; disabled legacy controls are excluded from CAC.

$$MDC = 100 \frac{\sum_{t \in T_{scope}} z_t}{|T_{scope}|}$$

MDC is the count-based closure of declared in-scope techniques; the published priority field is not used as a score weight.

$$CTIC = \frac{100}{4|I|} \sum_{i \in I} (q_i + h_i + a_i + m_i)$$

The flags represent lookup execution, MISP hit, Wazuh alert, and ATT&CK mapping. Initializing the lookup flag from an expectation field is not independent proof of query execution.

$$TF(\Delta, E) = \begin{cases} 100 & E = 1 \wedge \Delta \leq 60 \\ 80 & E = 1 \wedge 60 < \Delta \leq 1440 \\ 50 & E = 1 \wedge \Delta > 1440 \\ 0 & E = 0 \end{cases}$$

$E$  denotes the existence of at least one event, and  $\Delta$  is the age in minutes of the newest event across all sources at scoring time.  $TF$  does not measure per-source freshness.

#### C. Mathematical Definitions and Measurement Rationale

The SACI, CWLC, CAC, MDC, CTIC, and  $TF$  equations are scope-bounded operational measures introduced in this study, not off-the-shelf risk formulas reproduced from prior work. Separating numerators and denominators makes measurable scope, enabled-control state, and observed evidence independently auditable. This design is methodologically consistent with NIST SP 800-55 Volumes 1 and 2, which call for information-security measures to be specified with their purpose, scope, data, calculation, and interpretation, and with the monitoring, measurement, analysis, and evaluation principles of ISO/IEC 27004:2016 [26]–[28]. Those sources do not prescribe SACI's weights or freshness thresholds: the 0.30/0.25/0.20/0.15/0.10 weights and  $TF$  tiers are explicit study parameters. The weighted aggregation is an interpretable evidence-closure index, not a probability, expected-loss estimate, attack-risk estimate, or detection-accuracy measure.

#### D. Explainability Layer

The explainability layer presents component scores together with the asset, source, control, rule, technique, and CTI relationships that produced them. A reason-code mechanism can emit deterministic explanations for unclosed components, preventing the result from being reduced to one composite number.

An empty final reason-code output means that no scripted score gap was emitted in that run; it does not imply an absent explanation mechanism or the absence of data-integrity limitations. No LLM generates the numerical score; natural-language generation may only summarize verified tabular and graph evidence.

## E. Scoring Procedure

The pipeline declares the expected scope, constructs derived observation tables, computes five components, builds the typed graph, and publishes the results with integrity checks. The published scorer normalizes active metrics; the graph builder derives traceable relationship rows from the same coverage tables.

**Step 1: Define the evaluation scope with six assets and their expected asset-log-source pairs.**

**Step 2: Construct observation/flag tables for log sources, enabled controls, ATT&CK techniques, and two synthetic IOCs.**

**Step 3: Compute CWLC, CAC, MDC, CTIC, and TF using the implemented equations.**

**Step 4: Generate node and relationship rows while retaining observation state and evidence-provenance fields.**

**Step 5: Audit endpoint references, unique triples, isolated nodes, mapping consistency, and reason-code output.**

**Step 6: Renormalize active weights and publish SACI with the machine-readable research companion.**

## IV. CONTROLLED LABORATORY SETUP AND DATA PIPELINE

The experimental environment comprises six logical assets in the published 192.168.1.x address space. The topology links telemetry-producing endpoints to the central Wazuh collection and detection layer and then to ATT&CK, MISP, and SACI evidence relations.

**WSIEM (192.168.1.100): central Wazuh collection, decoder/rule evaluation, alerting, and custom-MISP-integration evidence.**

**DC01 (192.168.1.10): Windows Security and Sysmon; WS01 (192.168.1.111): Security, Sysmon, and PowerShell telemetry.**

**uhost (192.168.1.121): Ubuntu authentication, system-log, and process telemetry.**

**FW01-PFSENSE (192.168.1.1): firewall and pfSense syslog events.**

**CTI01 (192.168.1.101): MISP API, IOC, and enrichment evidence.**

**Evaluation universe: 6 assets, 12 asset-log pairs, 27 control rows, 25 enabled controls, 13 in-scope ATT&CK techniques, and 2 synthetic IOCs.**

### A. Evidence-Verified Topology and Setup Boundary

Because the release contains no environment-provisioning script, laboratory setup is reported at two levels: published asset, log, and path evidence establishes the verified experimental topology, while official product guides define the technical steps required to rebuild it. WSIEM is the central Wazuh node. Wazuh's official single-node quickstart describes server, indexer, and dashboard components, but the release does not establish that all three were co-located in the laboratory or identify a particular package version [14].

The telemetry flows for DC01, WS01, and uhost are consistent with Wazuh agents communicating with a central manager. A reconstruction must install agents for the relevant operating systems and enroll them with unique client keys [15], [16]. The published graph verifies collected\_by relations to Wazuh; it does not preserve the actual installation commands, ossec.conf files, certificates, hypervisor type, or exact product versions.

### B. Telemetry and Integration Configuration

The Windows layer observes Security, Microsoft-Windows-Sysmon/Operational, and Microsoft-Windows-PowerShell/Operational channels. Wazuh can collect these as eventchannel sources [17]; a Sysmon XML configuration provides process, network, and DNS visibility, while PowerShell Script Block Logging records command content in the Operational channel [22], [23]. Linux contributes authentication, syslog, and process evidence, and the network layer contributes pfSense remote-syslog evidence. Netgate documents remote logging and notes that unprotected UDP transport is unencrypted [21]. The release does not preserve the Sysmon XML, PowerShell policy, Linux forwarder configuration, or pfSense transport settings; these must not be presented as verified deployment facts.

CTI01 provides the threat-intelligence layer through the MISP API and two synthetic IOC records. Official MISP guides describe supported GNU/Linux installation paths and API-oriented operation [19], [20]. In the laboratory graph, Wazuh triggers a custom MISP integration, the integration queries MISP, an IOC match becomes a Wazuh alert, and the result is linked to ATT&CK context. This pattern is consistent with Wazuh Integrator and external-API connectors [18]. The MISP version, installation method, API endpoint, credentials, and custom connector configuration are not published.

### C. Functional Roles of the Tools and Technologies

Wazuh is the endpoint-event collection, decoder/rule evaluation, alerting, and ATT&CK-mapping layer [14]–[18]. Sysmon adds high-granularity Windows events such as process creation, network connections, and DNS activity, while PowerShell logging exposes command and script content [22], [23]. pfSense contributes network and firewall events through remote syslog [21]. MISP stores, queries, and shares IOCs and contextual threat intelligence through its API [19], [20]. Ansible with WinRM provides remote, repeatable orchestration of the Windows scenarios [24]. ATT&CK supplies the technique taxonomy, STIX supplies the machine-readable representation of threat-intelligence objects, and TAXII supplies an application-layer exchange protocol for that content [25], [29], [30]. Python scripts perform deterministic scoring and graph generation; the CYJS output supports interactive inspection in Cytoscape.js, while Mermaid provides text-defined, version-control-friendly diagrams [31], [32].

### D. Controlled Scenario Orchestration

The published attack runner executes only the DC01/Windows path. It operates under /opt/windows-control, reads inventory.ini, and targets dc01 with the Ansible win\_shell module. Ansible manages Windows

through WinRM with an appropriate authentication and encryption configuration [24]. The runner starts a benign Python HTTP server on the Wazuh host and produces account, domain, network, and share discovery, a DNS IOC lookup, and a benign file-transfer behavior. Each command is recorded as JSONL with scenario, ATT&CK, rule, and return-code fields.

After the final command, the runner waits 90 seconds for Wazuh ingestion and inspects `/var/ossec/logs/alerts/alerts.json` for the relevant custom-rule identifiers. Final coverage and alert evidence exists for Linux, pfSense, and CTI paths, but the research appendix does not publish equivalent provisioning or behavior-generation scripts for those paths. The experiment therefore does not claim the same reproduction level for the published Windows/DC01 orchestration and the outcome evidence of the other components.

### E. Data Collection, Scoring, and Publication

The scoring pipeline reads Wazuh alerts from `/var/ossec/logs/alerts/alerts.json`, MISP enrichment records from `/var/log/wazuh-misp/misp-enrichment.log`, normalized scenario events from `/var/log/saci/saci-scenario-events.log`, and pfSense evidence from `/var/log/saci/pfsense-events.log`. Asset and source attribution uses Wazuh agent name/identifier, provider, channel, event location, rule identifier, and ATT&CK metadata. Scope inventory, control definitions, technique scope, IOC expectations, and weights are loaded from CSV inputs under `/opt/saci-lab/data`.

The published Python scorer derives component and coverage tables, while the graph builder emits node/edge files and CYJS and Mermaid representations. ATT&CK technique identifiers can be validated against the official STIX/TAXII data services [25]. Because the score and graph derive from the same coverage tables, the graph is a provenance-oriented representation rather than an independent validation dataset. The package includes the runner, reason-code files, summary, source code, and validation manifest; the portal adds an explanatory layer over these immutable artifacts [13].

### F. Historical Sensitivity Suite

The final publication snapshot is the primary result. A separate S0-S18 suite contains 20 deterministic sensitivity scenarios spanning SIEM deployment, complete closure, and critical/noncritical telemetry, CTI, freshness, and rule loss. The S7 A/B branches produce 20 conditions under 19 labels. Historical S8's 95/173 graph is not merged with the final 97/171 graph because the two use different scopes and graph-generation pipelines.

### G. Reproducibility Boundary of the Setup

The released artifacts are sufficient to audit laboratory roles, IP addresses, observed data paths, formulas, and final outputs. They do not preserve Wazuh, MISP, pfSense, Sysmon, Windows, or Ubuntu versions; agent-enrollment commands; custom decoder/rule files; the Sysmon XML; PowerShell policies; pfSense forwarding settings; MISP API configuration; virtualization topology; or end-to-end provisioning automation. The official installation sources cited here therefore define the procedures required for an

independent rebuild rather than retroactively asserting unverified deployment details as laboratory facts.

## V. FINAL RESULTS AND DISCUSSION

In the final evaluation, 12/12 asset-log pairs are observed across six assets. Of 27 control rows, C003 and C004 remain disabled legacy controls; all 25 enabled controls are seen. All 13 in-scope techniques have covered=1, and all four implemented flags are 1 for each of two IOC records.

**CWLC: 100.0 (weight 0.30; 6 assets and 12/12 expected log pairs).**

**CAC: 100.0 (weight 0.25; 25/25 enabled controls; active weight 23.3/23.3).**

**MDC: 100.0 (weight 0.20; 13/13 declared in-scope techniques).**

**CTIC: 100.0 (weight 0.15; 2 IOC records x 4 flags = 8/8).**

**TF: 100 (weight 0.10; the globally newest event was in the ≤60-minute tier at scoring time).**

$$0.30 + 0.25 + 0.20 + 0.15 + 0.10 = 1.00$$
$$SACI = \frac{1.00 \times 100}{1.00} = 100.0$$

SACI=100 indicates that component expectations closed under the implemented rules within the declared final-publication evidence universe. It does not imply a secure environment, detection of every attack, coverage of the complete Enterprise ATT&CK knowledge base, or measurement of false-positive/negative rates.

The graph audit found 97 declared nodes, 171 edge rows with observed=1, 165 unique source-relationship-target triples, and 17 relationship labels. LOGSOURCE:Wazuh and MITRE:T1071.001 are undeclared endpoints; six declared nodes are isolated. Rule 110203 maps to T1071.001 in the control table but T1071.004 in the CTIC table. Thus, 171/171 row closure is not perfect referential integrity.

The historical sensitivity suite produced directionally expected changes: SACI rose from 0.00 in S0 to 94.00 after S6 CTI integration and 100.00 in historical S8 closure. Critical DC01 Sysmon loss yielded 78.53 in S7A versus 92.52 for noncritical WS01 loss in S7B; CTI loss yielded 92.50 in S12/S13 and freshness decay 95.00 in S15. S17 recovery returned to 100.00.

### A. Validity and Limitations

In construct-validity terms, SACI measures predefined evidence closure, not detection probability, prevention effectiveness, false-alarm rates, MTTD, or residual risk. Internal validity depends on expected rule/technique fields in the runner and scorer; some records can retain expected labels independently of runner return codes. The CTIC lookup flag is likewise expectation-derived.

External validity is limited to one six-asset Wazuh-MISP laboratory, two synthetic IOCs, and 13 selected techniques. TF uses the global newest event, so one fresh source can mask stale sources, and the exact scorer timestamp is not published. Because upstream `/opt/saci-lab/data` inputs, raw live-log snapshots, and complete environment automation are absent, the bundle supports arithmetic/graph audit rather than full end-to-end replay.

## VI. IMPLEMENTATION AND AUDITABILITY

The implementation comprises the published Windows scenario runner, deterministic Python scorer, and graph builder. The research appendix publishes 17 machine-readable files: coverage CSVs, score JSON/CSV, node/edge tables, CYJS/Mermaid graphs, reason-code outputs, and source code. Embedded SHA-256 checks verify release identity.

The interactive GitHub Pages portal presents methodology, architecture, final evidence tables, a scenario-selectable graph, and an artifact map [13]. It labels the final publication and historical scenarios separately, visualizes undeclared endpoints as explicit placeholders, and keeps the interpretation boundary visible.

## VII. AUDIT PROTOCOL AND OPERATIONAL INTERPRETATION

### A. Publication Layers and Audit Chain

The audit protocol separates three evidence layers: the declared evaluation scope, observation tables derived from that scope, and the published score/graph artifacts. The first layer fixes asset, source, control, technique, and IOC denominators; the second records observed, enabled, covered, and implementation flags; and the third carries component scores, relationship rows, and integrity findings. An independent reviewer can recompute numerators and denominators from the CSV and JSON files, reproduce the reduction of 171 edge rows to 165 unique triples, and compare undeclared endpoints against the node table. SHA-256 checks bind the identity of the publication package, but they do not by themselves establish the authenticity of upstream telemetry or prove that the run completed fully. Likewise, an empty reason-code output only means that the implemented rules emitted no explanation. SACI therefore treats deterministic package audit and complete experimental reproduction as distinct assurance levels.

### B. Decision Semantics of the Scores

Each evaluation should be read as a five-component score vector together with the active metric set, weights, and scope identifier, rather than as the composite SACI value alone. Active-weight normalization prevents an N/A component from being treated as an artificial zero; however, runs with different active metric sets have different denominators and are not directly comparable. The same requirement applies to versioning the scope, control states, and technique list. A documented policy may exclude disabled legacy controls from the CAC denominator, whereas an enabled but unseen control must remain a gap. Asset criticality also explains the designed asymmetry between S7A and S7B; that difference is not a probabilistic risk estimate. Under these conditions, SACI can serve as a publication gate, an evidence-completeness audit, or a fixed-scope longitudinal indicator. Ranking products, teams, or organizations across heterogeneous scopes would be methodologically unsupported without additional calibration and external validation.

## C. Sensitivity and Failure Semantics

The 20 scenarios in S0-S18 are deterministic perturbation and recovery experiments, not independent statistical observations. They test progression from an empty baseline through CTI integration and full closure, the unequal effect of critical and noncritical telemetry loss, removal of CTI flags, and decay across the freshness tiers. S7A falling below S7B confirms the intended criticality weighting; S12 and S13 converging at 92.50 confirms the same CTIC loss; and S15 decreasing to 95.00 reflects the expected TF contribution. These experiments show that the scorer responds to its inputs and implements the intended weight ordering. They do not estimate causal effect size or predictive performance because repeated trials, distributional assumptions, confidence intervals, and independent field samples are absent. Statistical validation would require a separate preregistered study with replicated runs, multiple laboratories, and observed operational outcomes.

## D. Productionization Principles

For production use, the pipeline should fail closed on unsuccessful runs, and every observation should be bound to a run identifier, an actual alert or event identifier, a source timestamp, and a file digest. Freshness should be computed per source rather than from the globally newest event; schema checks should reject missing or unexpected columns; and the graph builder should be able to invalidate a run under an explicit integrity policy, rather than merely report undeclared endpoints. The T1071.001/T1071.004 discrepancy for rule 110203 should be reconciled in one versioned mapping source. Scope, weight, and threshold configurations should accompany every result, while the portal should remain a derived view of the same immutable artifacts. Continuous-integration tests can enforce score-graph consistency, unique relationship keys, isolated-node policy, and reason-code coverage. External validity should ultimately be tested by independent teams applying the protocol across different SIEM products, network topologies, and operational sites.

## VIII. CONCLUSION

SACI demonstrates the feasibility of transforming a predefined SIEM-CTI evidence universe into a traceable graph and an active-weight-normalized visibility score. In the final evaluation, all five components score 100, arithmetically supported by 12/12 log pairs, 25/25 enabled controls, 13/13 in-scope techniques, and 8/8 CTI flags.

The result remains scope-bounded. Future work should require provenance based on actual alert identifiers and timestamps, exclude failed runner executions from scoring, reconcile the T1071.001/T1071.004 mapping, add per-source freshness, automate referential-integrity tests, and perform multisite external validation with complete environment and raw-data snapshots.

## IX. DATA AND ARTIFACT AVAILABILITY

The final score, coverage, reason-code, graph, and source-code artifacts, together with the interactive research companion, are published through the SACI Final Evidence Portal [13].

## REFERENCES

- [1] The MITRE Corporation, "Enterprise Matrix," MITRE ATT&CK. Accessed: Jun. 21, 2026. [Online]. Available: <https://attack.mitre.org/matrices/enterprise/>
- [2] Wazuh, "MITRE ATT&CK framework - Data analysis," Wazuh Documentation. Accessed: Jun. 21, 2026. [Online]. Available: <https://documentation.wazuh.com/current/user-manual/ruleset/mitre.html>
- [3] OASIS Cyber Threat Intelligence TC, "STIX Version 2.1," OASIS Standard, 2021. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>
- [4] K. Dempsey, N. Chawla, A. Johnson, R. Johnston, A. Jones, A. Orebaugh, M. Scholl, and K. Stine, "Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations," NIST Special Publication 800-137, 2011.
- [5] National Institute of Standards and Technology, "The NIST Cybersecurity Framework (CSF) 2.0," NIST Cybersecurity White Paper, 2024.
- [6] B. Al-Sada, A. Sadighian, and G. Oliveri, "MITRE ATT&CK: State of the Art and Way Forward," arXiv:2308.14016, 2023.
- [7] H. Manocha, A. Srivastava, C. Verma, R. Gupta, and B. Bansal, "Security Assessment Rating Framework for Enterprises using MITRE ATT&CK Matrix," arXiv:2108.06559, 2021.
- [8] K. Liu, F. Wang, Z. Ding, S. Liang, Z. Yu, and Y. Zhou, "A review of knowledge graph application scenarios in cyber security," arXiv:2204.04769, 2022.
- [9] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. IEEE Symposium on Security and Privacy, 2010, pp. 305-316.
- [10] V. Mavroeidis and S. Bromander, "Cyber threat intelligence model: An evaluation of taxonomies, sharing standards, and ontologies within cyber threat intelligence," in Proc. 2017 European Intelligence and Security Informatics Conference, 2017, pp. 91-98.
- [11] OASIS Cyber Threat Intelligence TC, "TAXII Version 2.1," OASIS Standard, 2021. [Online]. Available: <https://docs.oasis-open.org/cti/taxii/v2.1/taxii-v2.1.html>
- [12] M. Ussath, D. Jaeger, F. Cheng, and C. Meinel, "Advanced persistent threats: Behind the scenes," in Proc. 2016 Annual Conference on Information Science and Systems, 2016, pp. 181-186.
- [13] I. Arslan, "SACI Final Evidence Portal: research artifact," 2026. [Online]. Available: <https://4rslanismet.github.io/saci-final-evidence/> Accessed: Jul. 11, 2026.
- [14] Wazuh, "Quickstart," Wazuh Documentation. [Online]. Available: <https://documentation.wazuh.com/current/quickstart.html> Accessed: Jul. 11, 2026.
- [15] Wazuh, "Wazuh agent - Installation guide," Wazuh Documentation. [Online]. Available: <https://documentation.wazuh.com/current/installation-guide/wazuh-agent/index.html> Accessed: Jul. 11, 2026.
- [16] Wazuh, "Wazuh agent enrollment," Wazuh Documentation. [Online]. Available: <https://documentation.wazuh.com/current/user-manual/agent/agent-enrollment/index.html> Accessed: Jul. 11, 2026.
- [17] Wazuh, "Configuring log collection for different operating systems," Wazuh Documentation. [Online]. Available: <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/configuration.html> Accessed: Jul. 11, 2026.
- [18] Wazuh, "External API integration," Wazuh Documentation. [Online]. Available: <https://documentation.wazuh.com/current/user-manual/manager/integration-with-external-apis.html> Accessed: Jul. 11, 2026.
- [19] MISP Project, "Welcome to the official MISP Install Guides." [Online]. Available: <https://misp.github.io/MISP/> Accessed: Jul. 11, 2026.
- [20] MISP Project, "MISP Documentation and Support." [Online]. Available: <https://www.misp-project.org/documentation/> Accessed: Jul. 11, 2026.
- [21] Netgate, "Remote Logging with Syslog," pfSense Documentation. [Online]. Available: <https://docs.netgate.com/pfsense/en/latest/monitoring/logs/remote.html> Accessed: Jul. 11, 2026.
- [22] M. Russinovich and T. Garnier, "Sysmon v15.21," Microsoft Sysinternals, Jun. 17, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon> Accessed: Jul. 11, 2026.
- [23] Microsoft, "about Logging," Microsoft Learn - Windows PowerShell 5.1. [Online]. Available: [https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about\\_logging?view=powershell-5.1](https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging?view=powershell-5.1) Accessed: Jul. 11, 2026.
- [24] Ansible Project, "Windows Remote Management," Ansible Core Documentation. [Online]. Available: [https://docs.ansible.com/projects/ansible-core/2.17/os\\_guide/windows\\_winrm.html](https://docs.ansible.com/projects/ansible-core/2.17/os_guide/windows_winrm.html) Accessed: Jul. 11, 2026.
- [25] MITRE, "ATT&CK Data & Tools," MITRE ATT&CK. [Online]. Available: <https://attack.mitre.org/resources/attack-data-and-tools/> Accessed: Jul. 11, 2026.
- [26] K. Schroeder, H. Trinh, and V. Pillitteri, "Measurement Guide for Information Security: Volume 1—Identifying and Selecting Measures," NIST SP 800-55 Vol. 1, Dec. 2024, doi: 10.6028/NIST.SP.800-55v1.
- [27] K. Schroeder, H. Trinh, and V. Pillitteri, "Measurement Guide for Information Security: Volume 2—Developing an Information Security Measurement Program," NIST SP 800-55 Vol. 2, Dec. 2024, doi: 10.6028/NIST.SP.800-55v2.
- [28] ISO/IEC, "ISO/IEC 27004:2016—Information technology—Security techniques—Information security management—Monitoring, measurement, analysis and evaluation," 2nd ed., Dec. 2016. [Online]. Available: <https://www.iso.org/standard/64120.html> Accessed: Jul. 12, 2026.
- [29] B. Jordan, R. Piazza, and T. Darley, eds., "STIX Version 2.1," OASIS Standard, Jun. 10, 2021. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html> Accessed: Jul. 12, 2026.
- [30] B. Jordan, R. Piazza, and T. Darley, eds., "TAXII Version 2.1," OASIS Standard, Jun. 10, 2021. [Online]. Available: <https://docs.oasis-open.org/cti/taxii/v2.1/os/taxii-v2.1-os.html> Accessed: Jul. 12, 2026.
- [31] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, and G. D. Bader, "Cytoscape.js: a graph theory library for visualisation and analysis," *Bioinformatics*, vol. 32, no. 2, pp. 309–311, 2016, doi: 10.1093/bioinformatics/btv557.
- [32] Mermaid, "About Mermaid," Mermaid Documentation. [Online]. Available: <https://mermaid.js.org/intro/> Accessed: Jul. 12, 2026.